

Technische Hochschule Mittelhessen
Fachbereich MNI
Praktikum Wirtschaftsinformatik, Sommersemester 2026

Aufgabendatenbank

Projektdokumentation

Betreuung: Prof. Dr. Markus Siepermann; Johannes Kunz
Projektteam: [Name 1] (Projektleitung)
[Name 2]
[Name 3]
Gruppe: [Gruppennummer]
Abgabe: 14. Juli 2026

Inhaltsverzeichnis

1 Kurze Zusammenfassung

Ausgangspunkt war ein vorgegebenes relationales Schema zur Beschreibung und Organisation von Übungsaufgaben. Dieses abstrakte Modell wurde in eine nutzbare Webanwendung aus Vue-Frontend, Spring-Boot-Backend und PostgreSQL-Datenbank überführt.

Die Anwendung unterstützt Aufgaben mit mehreren Inhaltsbausteinen und Metadaten, geordnete Sequenzen, ungeordnete Sammlungen und horizontale Varianten. Hinzu kommen hierarchische Tags, kombinierte Such- und Filterfunktionen, Validatoren und konfigurierbare Darstellungstemplates. Docker, automatisierte Tests und eine Build-Pipeline nach GHCR schaffen eine reproduzierbare technische Grundlage für die weitere Plattformintegration.

2 Aufgabenstellung und Zielsetzung

Grundlage war kein frei entworfenes Datenmodell, sondern ein bereits abgestimmtes Schema mit Entitäten wie `item`, `item_content`, `item_collection`, `tag` und `validator`. Das Team musste dessen fachliche Bedeutung verstehen, technisch umsetzen und in eine verständliche Oberfläche übersetzen.

Ziel war ein Autorenwerkzeug, mit dem Lehrende Aufgaben beschreiben, strukturieren, wiederfinden und kombinieren können. Die Anwendung ist kein Lern-, Abgabe- oder Korrektursystem. Validatoren und Modifikatoren werden gespeichert; ihre Ausführung gehört zu einem externen Dienst. In der ersten Iteration dürfen alle Nutzenden Aufgaben anlegen und bearbeiten. Eine zentrale Authentifizierung kann später ergänzt werden.

3 Ausgangssituation und Recherche

Zu Projektbeginn lagen das Schema und eine grobe fachliche Beschreibung vor. Zunächst waren daher folgende Fragen zu klären:

- Wie werden Aufgaben von ihren Inhaltsbausteinen getrennt?
- Wie unterscheiden sich Sequenzen, Sammlungen und Varianten?
- Welche Regeln führt die Anwendung aus und welche speichert sie nur?
- Wie werden hierarchische Metadaten verständlich bedienbar?

Aus Schemaanalyse und Betreuerabstimmungen entstand insbesondere die Trennung zwischen vertikalen Beziehungen über Collections und horizontalen Varianten über `root_item_id`.

3.1 Technologieauswahl

Bereich	Technologie	Begründung
Frontend	Vue 3, TypeScript, Vuetify, Pinia	Komponentenbasierte Oberfläche, Typsicherheit, zentraler Zustand
Backend	Java 21, Spring Boot, Spring Data JPA	Klare Schichten, Transaktionen, Abbildung des Schemas
Datenbank	PostgreSQL 16	UUID, JSONB und BYTEA
Tests	Vitest, JUnit, Mockito, MockMvc	Automatisierte Prüfung von Frontend-, Service- und Controllerlogik
Betrieb	Docker, GitHub Actions, GHCR, Kubernetes	Reproduzierbarer Start und Plattformvorbereitung

Tabelle 1: Technischer Lösungsrahmen

4 Anforderungen und Rahmenbedingungen

4.1 Funktionale Anforderungen

F1 Aufgaben mit Autor, Lizenz und Typ erstellen und bearbeiten.

F2 Inhaltsbausteine mit Zweck, Typ und Lizenz verwalten.

F3 Text, Bilder und PDF-Dateien speichern und darstellen.

F4 Geordnete Sequenzen und ungeordnete Sammlungen bilden.

F5 Varianten einer Ausgangsaufgabe verwalten.

F6 Validatoren definieren und Aufgaben zuordnen.

F7 Hierarchische Tags anlegen, zuordnen und filtern.

F8 Nach Inhalt, Autor, Typ und Tags suchen.

F9 Darstellungstemplates bearbeiten und als Vorschau anwenden.

4.2 Nicht-funktionale Anforderungen

- referentielle Integrität und transaktionale Backend-Operationen,
- typisierte Verträge zwischen Frontend und Backend,
- verständliche Bedienung trotz abstraktem Datenmodell,
- modularer Aufbau und automatisierte Tests,
- reproduzierbarer Start in Containern,
- keine strukturelle Erweiterung des vorgegebenen Schemas.

5 Konzept und Entwurf

5.1 Systemarchitektur

Das Vue-Frontend kommuniziert über REST mit dem Spring-Boot-Backend. Dieses ist in Controller, Services und Spring-Data-Repositories gegliedert und greift auf PostgreSQL zu. Im Frontend spricht der Store gegen das Interface **ApiAdapter**. Eine HTTP-Implementierung und ein Dummy-Adapter erlauben den Betrieb mit realem Backend oder lokalen Entwicklungsdaten.

Vue-Komponenten -> Pinia Store -> ApiAdapter -> Controller
-> Service
-> Repository -> PostgreSQL

5.2 Datenmodell

Item. Ein Item ist die zentrale fachliche Einheit mit Autor, Lizenz, Typ und optionalem Darstellungstemplate. `root_item_id` verbindet Varianten mit ihrer Ausgangsaufgabe.

ItemContent. Inhalte sind vom Item getrennt. JSONB speichert strukturierte Inhalte, BYTEA Binärdaten. `item_contents` ordnet einen Inhalt einem Item zu und ergänzt den `purpose`, beispielsweise Aufgabenstellung, Hinweis oder Lösung.

ItemCollection. Eine Collection referenziert über `parent_item_id` ein Item als fachlichen Träger, besitzt aber eine eigene technische ID. Mitglieder liegen in `item_collection_sub_item`. Geordnete Collections besitzen fortlaufende Positionen; bei ungeordneten Collections ist die Position NULL.

Tags und Regeln. Tags bilden über `parent_tag_id` eine Hierarchie. Validatoren und Modifikatoren sind über Join-Tabellen zugeordnet.

5.3 Beziehungsmodell

	Vertikal	Horizontal
Bedeutung	Lernpfad oder Gruppierung verschiedener Aufgaben	Varianten derselben Ausgangsaufgabe
Umsetzung	Collection, Sub-Items und optional <code>position</code>	<code>root_item_id</code> , Validatoren/Modifikatoren
Reihenfolge	optional	keine

Tabelle 2: Trennung der Beziehungsarten

6 Umsetzung

6.1 Aufgaben und Inhaltsbausteine

Eine Aufgabe wird über einen bestätigungspflichtigen Dialog erstellt. Nach der Auswahl von Autor, Lizenz und Typ wird das Item angelegt und mit seinen Inhaltsbausteinen verbunden. Die Oberfläche aktualisiert sich optimistisch. Schlägt ein mehrstufiger Vorgang teilweise fehl, wird der betroffene Teilbaum erneut geladen, damit Frontend und Datenbank konvergieren.

Inhalte können als Text, Bild oder PDF gespeichert werden. Jeder Baustein hat einen Zweck und eigene Metadaten. Darstellungstemplates bestimmen die Reihenfolge der Zwecke in der Vorschau.

6.2 Collections und Drag-and-Drop

Collections erscheinen als Ordner. Ungeordnete Collections gruppieren Aufgaben thematisch; geordnete Collections bilden Sequenzen. Das Backend verwaltet ihre Positionen. Aufgaben können innerhalb und zwischen Collections verschoben werden. Item- und Collection-ID bleiben dabei getrennt. Die Umwandlung eines Items in eine Collection ist idempotent und erzeugt bei wiederholter Anfrage keinen zweiten Collection-Datensatz.

6.3 Varianten und Validatoren

Varianten referenzieren die Ausgangsaufgabe über `root_item_id`. Diese Referenz wird nicht für Collection-Hierarchie verwendet. Validatoren beschreiben Restriktionen und können erstellt, bearbeitet und Aufgaben zugeordnet werden. Ihre Ausführung bleibt einem externen Dienst vorbehalten.

6.4 Hierarchische Tags und Suche

Tags werden als Baum und als lesbare Pfade dargestellt. Sie können in der Anwendung angelegt und Aufgaben zugeordnet werden. Die Filterleiste verbindet Volltextsuche mit Filtern nach Autor, Aufgabentyp und Tags.

6.5 Qualitätssicherung

Frontend-Tests prüfen Store-Aktionen, Typ-Hilfsfunktionen, Baumvalidierung, Collections, Varianten, Suche und die Resynchronisation nach partiellen API-Fehlern. Backend-Tests decken Item-, Content-, Validator-, Template- und Collection-Services ab; MockMvc prüft zentrale Endpunkte und HTTP-Statuscodes. Ergänzende Ende-zu-Ende-Tests mit realer PostgreSQL-Instanz bleiben eine sinnvolle Weiterentwicklung.

6.6 Bereitstellung

Docker Compose startet Datenbank, Backend und Frontend gemeinsam. GitHub Actions baut Container-Images und veröffentlicht sie in GHCR. `values.yaml` bereitet die Kubernetes-Bereitstellung vor. Das vollständige Ausrollen hängt zusätzlich von Infrastruktur und zentralen Plattformdiensten ab.

7 Projektorganisation

7.1 Vorgehen und Versionsverwaltung

Die Entwicklung erfolgte über Feature-Branches und Pull-Requests. Da Store, API-Adapter und DTOs von mehreren Funktionen betroffen waren, entstanden zeitweise Merge-Konflikte. Häufigere Integration, Typprüfung und Tests reduzierten dieses Risiko.

7.2 Projektphasen und Meilensteine

Phase	Zeitraum	Ergebnis
Analyse	Ende April–Mai	Schemaanalyse, Abgrenzung, Technologieauswahl
Grundaufbau	Mai	Datenbank, Backend, Frontend-Grundstruktur, Docker
Kernfunktionen	Juni	Items, Contents, Collections, API-Integration
Erweiterung	Juni–Juli	Varianten, Validatoren, Templates, Tags, Suche
Stabilisierung	Juli	Tests, Fehlerbehandlung, CI/CD, Dokumentation
Abgabe	14. Juli	Bereinigter Stand und Dokumentation

Tabelle 3: Projektphasen

7.3 Aufgabenverteilung

Person	Nachweisbarer Schwerpunkt
[Name 1]	[Projektleitung, Koordination und konkrete Beiträge]
[Name 2]	[Konkrete Frontend-, Backend- oder Datenbankbeiträge]
[Name 3]	[Konkrete Frontend-, Test- oder Deploymentbeiträge]

Tabelle 4: Aufgabenverteilung

Die Tabelle ist vor Abgabe anhand der Pull-Requests zu konkretisieren.

7.4 Abweichungen vom ursprünglichen Plan

Das Team wurde von fünf auf drei Personen reduziert. Aufgaben mussten neu priorisiert und schichtübergreifend verteilt werden. Unklare Konzepte wie Templates, Varianten und Modifier wurden zunächst fachlich geklärt.

Die Ausführung von Validatoren und Modifikatoren wurde nicht aufgenommen, da sie zu einem externen Laufzeitdienst gehört. Cluster-Deployment und zentrale Authentifizierung hängen von der Plattform ab; die notwendigen Container, Schnittstellen und Konfigurationen wurden vorbereitet.

8 Fazit und Ausblick

Das Projekt zeigt, wie ein abstraktes Schema in ein nutzbares Autorenwerkzeug überführt werden kann. Die Trennung von Aufgaben und Inhalten, Collections und Varianten sowie die Bedienung hierarchischer Metadaten gehen über ein einfaches CRUD-System hinaus und unterstützen konkrete Arbeitsabläufe.

Weitere Schritte sind Ende-zu-Ende-Tests mit PostgreSQL, die Modularisierung des zentralen Stores, eine abschließende Präzisierung der Mehrfachzuordnung von Items und Contents sowie die zentrale Authentifizierung. Architektur und Containerisierung bilden dafür eine belastbare Grundlage.

Literatur

- [1] Vue.js Documentation. <https://vuejs.org/>
- [2] Pinia Documentation. <https://pinia.vuejs.org/>

- [3] Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- [4] PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- [5] Docker Documentation. <https://docs.docker.com/>
- [6] Kubernetes Documentation. <https://kubernetes.io/docs/>
- [7] THM E-Learning-Plattform: interne Architektur- und Onboarding-Unterlagen.

A Anhang

A.1 Lokale Ausführung

```
docker compose up --build
# Frontend: http://localhost:8085
# Backend:  http://localhost:8080
# Dummy:    cd frontend && npm run dev:dummy
```

A.2 Vor Abgabe zu ergänzen

- Namen, Gruppennummer und konkrete Aufgabenverteilung,
- fünf zugeschnittene Anwendungsscreenshots,
- lesbares ER-Diagramm,
- endgültiger Repository-Link,
- Datum des letzten erfolgreichen Build- und Testlaufs.